MAST32001 Computational Statistics - Lecture 11

Gibbs Sampling, HMC/NUTS and Stan

Klaus Nordhausen

University of Helsinki

WS 2026

Gibbs sampling

Gibbs sampling idea

Suppose the target is a joint density

$$\pi(\theta_1, \ldots, \theta_p).$$

If the full conditional distributions

$$\pi(\theta_j \mid \theta_{-j})$$

are easy to sample from, update one component at a time.

For two parameters:

$$\theta_1^{(t+1)} \sim \pi(\theta_1 \mid \theta_2^{(t)}),$$

then

$$\theta_2^{(t+1)} \sim \pi(\theta_2 \mid \theta_1^{(t+1)}).$$

No accept/reject step is needed.

Why Gibbs leaves the target invariant

Suppose

$$(\theta_1, \theta_2) \sim \pi(\theta_1, \theta_2).$$

If we keep θ_2 and redraw

$$\theta_1' \sim \pi(\theta_1 \mid \theta_2),$$

then

$$p(\theta_1', \theta_2) = \pi(\theta_1' \mid \theta_2)\pi(\theta_2) = \pi(\theta_1', \theta_2).$$

Thus a full-conditional update preserves the joint target.

Composing such updates also preserves the target.

Gibbs as a special Metropolis–Hastings step

For an update of θ_1, choose

$$q(\theta_1' \mid \theta_1, \theta_2) = \pi(\theta_1' \mid \theta_2).$$

The Metropolis–Hastings ratio becomes 1.

So Gibbs sampling can be viewed as a sequence of MH updates with acceptance probability 1.

Gibbs example

Normal model with unknown mean and variance

Suppose

$$Y_i \mid \mu, \sigma^2 \overset{iid}{\sim} N(\mu, \sigma^2).$$

Use

$$\mu \sim N(m_0, s_0^2)$$

and

$$\sigma^2 \sim \text{InvGamma}(a_0, b_0),$$

independently.

The likelihood is

$$L(\mu, \sigma^2; y) \propto (\sigma^2)^{-n/2} \exp\left\{ -\frac{1}{2\sigma^2} \sum_{i=1}^{n} (y_i - \mu)^2 \right\}.$$

Derive the conditional for μ

Condition on σ^2. The kernel in μ is

$$\exp\left\{-\frac{1}{2}\left[\frac{1}{\sigma^2}\sum_i (y_i-\mu)^2 + \frac{(\mu-m_0)^2}{s_0^2}\right]\right\}.$$

Collecting terms in μ,

$$\mu \mid \sigma^2, y \sim N(m_n, s_n^2),$$

where

$$s_n^2 = \left(\frac{n}{\sigma^2} + \frac{1}{s_0^2}\right)^{-1}$$

and

$$m_n = s_n^2\left(\frac{n\bar{y}}{\sigma^2} + \frac{m_0}{s_0^2}\right).$$

Derive the conditional for σ^2

Condition on μ. Combine

$$(\sigma^2)^{-n/2} \exp\left\{ -\frac{\sum_i (y_i - \mu)^2}{2\sigma^2} \right\}$$

with the inverse-Gamma prior kernel

$$(\sigma^2)^{-(a_0+1)} \exp\left\{ -\frac{b_0}{\sigma^2} \right\}.$$

Hence

$$\sigma^2 \mid \mu, y \sim \text{InvGamma}\left(a_0 + \frac{n}{2},\ b_0 + \frac{1}{2} \sum_i (y_i - \mu)^2 \right).$$

Full Gibbs sampler in R

Base R has no inverse-Gamma RNG, but if $S \sim \text{InvGamma}(a, b)$, then $1/S \sim \text{Gamma}(a, b)$.

```r
gibbs_normal <- function(y, n_iter, m0 = 0, s0 = 10,
                         a0 = 2, b0 = 2, mu_init = 0,
                         sigma2_init = 1) {
  n <- length(y)
  mu <- numeric(n_iter)
  sigma2 <- numeric(n_iter)
  mu[1] <- mu_init
  sigma2[1] <- sigma2_init

  for (t in 2:n_iter) {
    s2_n <- 1 / (n / sigma2[t - 1] + 1 / s0^2)
    m_n <- s2_n * (sum(y) / sigma2[t - 1] + m0 / s0^2)
    mu[t] <- rnorm(1, m_n, sqrt(s2_n))
    a_n <- a0 + n / 2
    b_n <- b0 + 0.5 * sum((y - mu[t])^2)
    sigma2[t] <- 1 / rgamma(1, shape = a_n, rate = b_n)
  }
  cbind(mu = mu, sigma2 = sigma2)
}
```

Run and inspect

```r
set.seed(201)

y <- rnorm(50, mean = 2, sd = 1.5)
gibbs_draws <- gibbs_normal(
  y, n_iter = 12000, mu_init = -5, sigma2_init = 10
)
gibbs_keep <- gibbs_draws[-seq_len(2000), ]
apply(gibbs_keep, 2, mean)
```

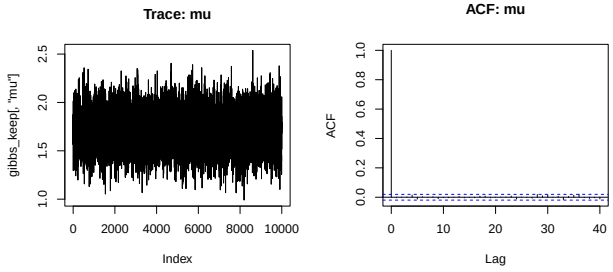
```
##       mu   sigma2
## 1.715865 1.762550
```

```r
apply(gibbs_keep, 2, quantile, c(0.025, 0.5, 0.975))
```

```
##             mu   sigma2
## 2.5%  1.347730 1.193031
## 50%   1.716685 1.716835
## 97.5% 2.087662 2.562688
```

Run and inspect

```
par(mfrow = c(1, 2))
plot(gibbs_keep[, "mu"], type = "l", main = "Trace: mu")
acf(gibbs_keep[, "mu"], main = "ACF: mu")
```




```
par(mfrow = c(1, 1))
```

Interactive 1: derive before coding

Keep the same normal likelihood but change the prior to

$$\mu \mid \sigma^2 \sim N\left(m_0, \frac{\sigma^2}{\kappa_0}\right).$$

Here $\kappa_0 > 0$ controls the prior precision: larger κ_0 means stronger shrinkage toward m_0.

In pairs:

1. write the kernel of $p(\mu \mid \sigma^2, y)$;
2. derive its mean and variance;
3. compare the result with the independent prior used above;
4. predict whether this parameterization might make Gibbs updates more or less coupled.

The goal is to practice deriving **full conditionals**, not memorizing them.

Interactive 1: solution

With

$$y_i \mid \mu, \sigma^2 \sim N(\mu, \sigma^2), \qquad \mu \mid \sigma^2 \sim N\left(m_0, \frac{\sigma^2}{\kappa_0}\right),$$

the conditional kernel is

$$p(\mu \mid \sigma^2, y) \propto \exp\left\{-\frac{1}{2\sigma^2}\left[\sum_{i=1}^{n}(y_i - \mu)^2 + \kappa_0(\mu - m_0)^2\right]\right\}.$$

Collecting the quadratic terms in μ gives

$$\mu \mid \sigma^2, y \sim N\left(\frac{n\bar{y} + \kappa_0 m_0}{n + \kappa_0}, \frac{\sigma^2}{n + \kappa_0}\right).$$

Interactive 1: solution – interpretation

The posterior mean is a precision-weighted compromise:

$$\frac{n}{n+\kappa_0}\bar{y} + \frac{\kappa_0}{n+\kappa_0}m_0.$$

Compared with the independent prior $\mu \sim N(m_0, s_0^2)$, the conditional prior variance now scales with σ^2.

Therefore the updates of μ and σ^2 are more explicitly coupled: changing σ^2 changes the conditional variance of μ.

This parameterization is conjugate and analytically convenient, but **conjugacy does not automatically imply fast Gibbs mixing**.

When Gibbs struggles

Correlated targets

Gibbs sampling can be inefficient when parameters are strongly correlated.

Imagine a narrow tilted bivariate normal density.

Updating one coordinate at a time forces the chain to move in a zig-zag pattern.

The chain is valid but may have large autocorrelation and small effective sample size.

Parameterization can matter as much as the choice of algorithm.

Interactive 2: correlated bivariate normal

Let

$$\begin{pmatrix} X \\ Y \end{pmatrix} \sim N\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \right).$$

The full conditionals are

$$X \mid Y = y \sim N(\rho y, 1 - \rho^2),$$
$$Y \mid X = x \sim N(\rho x, 1 - \rho^2).$$

Run the following Gibbs sampler for $\rho = 0.2$ and $\rho = 0.99$. Compare trace plots and ACFs.

Interactive 2: correlated bivariate normal

```r
gibbs_bvn <- function(rho, n_iter = 10000) {
  z <- matrix(0, n_iter, 2)

  for (t in 2:n_iter) {
    z[t, 1] <- rnorm(1, mean = rho * z[t - 1, 2],
      sd = sqrt(1 - rho^2)
    )
    z[t, 2] <- rnorm(1, mean = rho * z[t, 1],
      sd = sqrt(1 - rho^2)
    )
  }
  z
}
```

Question:

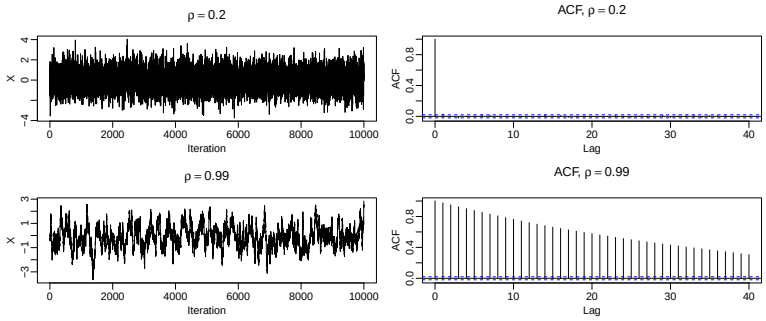
Can a perfectly valid MCMC algorithm still be computationally poor?

Interactive 2: solution – run both chains

```
set.seed(20260812)
z_02 <- gibbs_bvn(rho = 0.2)
z_099 <- gibbs_bvn(rho = 0.99)

par(mfrow = c(2, 2), mar = c(2.8, 2.8, 3.2, 0.8), mgp = c(1.6, 0.5, 0))
plot(z_02[, 1], type = "l", main = expression(rho == 0.2),
     xlab = "Iteration", ylab = "X")
acf(z_02[, 1], main = expression(paste("ACF, ", rho == 0.2)))
plot(z_099[, 1], type = "l", main = expression(rho == 0.99),
     xlab = "Iteration", ylab = "X")
acf(z_099[, 1], main = expression(paste("ACF, ", rho == 0.99)))
par(mfrow = c(1, 1))
```

Interactive 2: solution – run both chains



Both samplers are valid, but their computational efficiency is very different.

Interactive 2: solution – why?

For $\rho = 0.2$,

$$X \mid Y = y \sim N(0.2y, 0.96),$$

so a coordinate update can move substantially.

For $\rho = 0.99$,

$$X \mid Y = y \sim N(0.99y, 0.0199),$$

so each conditional distribution is extremely narrow. The sampler therefore moves only a small distance across the long diagonal direction of the joint distribution.

Hence:

valid stationary distribution $\neq$ efficient exploration.

This is exactly what the trace plots and slowly decaying ACF reveal.

Systematic scan and random scan

The standard Gibbs sampler updates

$$\theta_1, \theta_2, \ldots, \theta_p$$

in a fixed order. This is called **systematic scan**.

An alternative is to choose coordinates randomly.

Both can have the correct invariant distribution, but their convergence properties need not be identical.

The important point is that a Gibbs sampler is itself a Markov chain whose transition kernel is built by composing conditional updates.

Blocking

Instead of updating one scalar at a time, we may update a block:

$$\theta_A \sim \pi(\theta_A \mid \theta_{-A}).$$

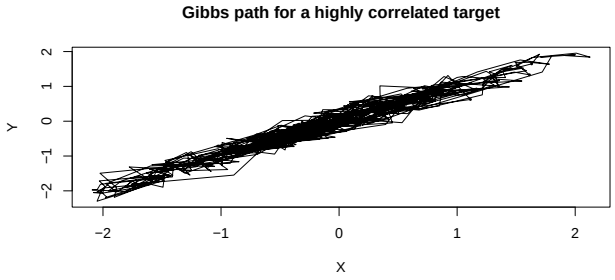
Blocking can greatly improve mixing when strongly related parameters can be sampled jointly.

But large blocks may be computationally difficult.

Thus there is a tradeoff:

easy conditional updates versus efficient movement through the joint target.

Correlation and Gibbs:



The target is a long diagonal ellipse, but coordinate-wise updates create slow zig-zag movement.

Hamiltonian Monte Carlo

Why another MCMC method?

Random-walk Metropolis explores locally.

In moderate or high dimensions this can lead to:

- many rejected proposals;
- small accepted steps;
- slow diffusion through the target.

Hamiltonian Monte Carlo uses **gradients of the log target** to propose distant states while staying in regions of high probability.

The price is more computation per proposal.

Augment with momentum

Let the target be

$$\pi(q) \propto e^{-U(q)}, \qquad U(q) = -\log \pi(q) + C.$$

Introduce momentum

$$p \sim N(0, M).$$

Define kinetic energy

$$K(p) = \frac{1}{2} p^\top M^{-1} p$$

and Hamiltonian

$$H(q, p) = U(q) + K(p).$$

The joint density is

$$\pi(q, p) \propto e^{-H(q,p)}.$$

Marginalizing out p returns the desired target for q.

Hamilton's equations

Hamiltonian dynamics satisfy

$$\frac{dq}{dt} = \frac{\partial H}{\partial p} = M^{-1}p,$$

$$\frac{dp}{dt} = -\frac{\partial H}{\partial q} = -\nabla U(q).$$

Exact Hamiltonian dynamics preserve $H(q, p)$ **exactly**. Because

$$\pi(q, p) \propto e^{-H(q,p)},$$

they therefore move along contours of **constant joint density**.

This is the key geometric idea: the dynamics can travel a long distance through parameter space without having to cross into much lower joint density.

Exact dynamics versus numerical HMC

There are three distinct ideas to keep separate:

1. **Exact Hamiltonian dynamics:** $H(q, p)$ is conserved exactly, so the joint density is constant along the trajectory.
2. **Leapfrog integration:** in practice we approximate the dynamics numerically, so $H(q, p)$ is only approximately conserved.
3. **Metropolis correction:** the final accept/reject step corrects the numerical error and preserves the desired target distribution.

Thus the word *approximately* belongs to the **leapfrog trajectory**, not to the exact Hamiltonian dynamics.

Leapfrog integrator

Exact Hamiltonian trajectories are usually unavailable in closed form. HMC therefore uses the **leapfrog integrator** to approximate them.

For step size ϵ:

$$p_{t+1/2} = p_t - \frac{\epsilon}{2}\nabla U(q_t),$$

$$q_{t+1} = q_t + \epsilon M^{-1} p_{t+1/2},$$

$$p_{t+1} = p_{t+1/2} - \frac{\epsilon}{2}\nabla U(q_{t+1}).$$

After L leapfrog steps, an MH correction accepts the numerical trajectory with probability

$$\min\{1, \exp[-H(q', p') + H(q, p)]\}.$$

HMC tuning problem

Basic HMC requires choices of:

- step size ϵ;
- number of leapfrog steps L;
- mass matrix M.

Too large ϵ gives inaccurate trajectories and rejections.

Too small ϵ wastes computation.

Too short a trajectory gives random-walk-like exploration.

Too long a trajectory can turn back and waste computation.

HMC intuition: use momentum instead of a random walk

Think of q as position on a landscape with height

$$U(q) = -\log \pi(q) + C.$$

Random-walk Metropolis repeatedly proposes nearby positions without using the slope.

HMC introduces momentum and follows the local gradient of the landscape.

This allows coherent movement across long distances while remaining close to the typical high-probability region.

A Gaussian target as a physical system

For a one-dimensional standard normal target,

$$\pi(q) \propto e^{-q^2/2},$$

so

$$U(q) = \frac{1}{2}q^2.$$

With unit mass,

$$K(p) = \frac{1}{2}p^2,$$

and therefore

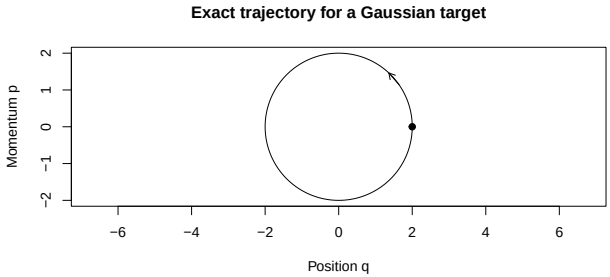
$$H(q,p) = \frac{1}{2}(q^2 + p^2).$$

Contours of constant Hamiltonian satisfy

$$q^2 + p^2 = \text{constant}.$$

Exact Hamiltonian dynamics move around these circles rather than diffusing randomly.

Hamiltonian trajectory



For this idealized target, energy is exactly conserved.

- The circle lives in the **augmented position–momentum space** (q, p), not in the original parameter space.
- As the system moves, **potential energy** $U(q)$ and **kinetic energy** $K(p)$ are exchanged, while their sum $H(q, p)$ stays constant.
- There is no friction: the trajectory does not settle at the mode. The artificial momentum is refreshed between HMC trajectories to explore the posterior.

Why leapfrog?

A numerical integrator for HMC should approximately preserve the geometry of Hamiltonian dynamics.

Leapfrog is:

- reversible;
- volume preserving;
- second-order accurate;
- computationally simple.

These properties make the final Metropolis correction possible without computing a Jacobian determinant.

The acceptance step corrects the energy error introduced by numerical integration. With a well-tuned step size this error is typically small, so acceptance can remain high.

Step size and energy error

With a very large step size, leapfrog no longer follows the Hamiltonian trajectory accurately.

Then

$$\Delta H = H(q', p') - H(q, p)$$

can become large, leading to low acceptance or numerical instability.

With a very small step size, trajectories are accurate but expensive.

This is why HMC needs step-size adaptation.

The typical set

In high dimensions, the mode is often not where most probability mass lies.

For example, if

$$X \sim N_d(0, I),$$

the density is largest at 0, but the radius

$$\|X\|$$

is typically around $\sqrt{d}$.

HMC is best understood as exploring the **typical set**, not as moving toward the mode.

This is another reason optimization intuition can be misleading for MCMC.

NUTS: what is a U-turn?

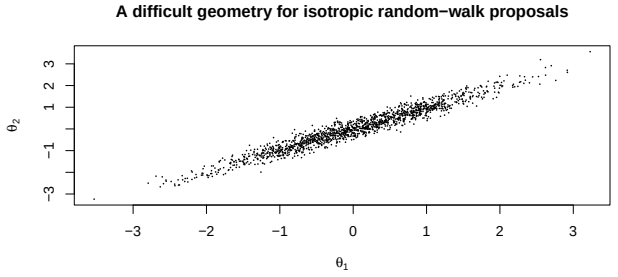
A Hamiltonian trajectory initially moves away from its starting point.

Eventually it may begin to double back, so further integration wastes computation.

NUTS builds a trajectory adaptively and uses a geometric U-turn criterion to decide when continued expansion is no longer useful.

The exact algorithm is more subtle than simply “stop and take the endpoint”: it constructs and samples from a set of candidate states while preserving the target distribution.

HMC versus random-walk MH: geometry



Gradient information helps HMC follow the geometry rather than repeatedly proposing across the narrow direction.

NUTS and probabilistic programming

What is Stan?

Stan is a probabilistic programming system for Bayesian statistical modeling and computation.

Instead of implementing an MCMC transition rule ourselves, we specify

- the observed data;
- the unknown parameters;
- the statistical model: priors and likelihood.

Stan then constructs the log posterior and its derivatives using automatic differentiation and can sample from the posterior using HMC/NUTS.

RStan is the R interface: R supplies the data and calls Stan; Stan performs the sampling; the posterior draws return to R for summaries and diagnostics.

The important distinction is

we specify the model; Stan handles much of the MCMC computation.

No-U-Turn Sampler

NUTS is an adaptive variant of HMC.

Conceptually, it:

- builds a Hamiltonian trajectory dynamically;
- stops expanding when the trajectory begins to turn back on itself;
- adapts the step size during warmup;
- in Stan, can also adapt the metric/mass-matrix representation.

Thus the user does not need to choose a fixed number of leapfrog steps.

Stan's default and preferred MCMC algorithm is NUTS.

What Stan does

Stan lets us specify a probabilistic model and then performs inference using gradient-based algorithms.

RStan is the R interface to Stan and supports Bayesian inference with NUTS/HMC.

The computational workflow is:

$$\boxed{\text{data} + \text{model} \rightarrow \text{log posterior} + \text{gradients} \rightarrow \text{NUTS draws}}$$

Stan uses automatic differentiation to obtain the gradients needed by HMC/NUTS.

Minimal example: first specify the model

We continue with the simulated normal data used in the Gibbs example:

$$Y_i \mid \mu, \sigma \sim N(\mu, \sigma^2).$$

This is the **likelihood model**. To obtain a Bayesian model we also need priors. For illustration choose

$$\mu \sim N(0, 10^2), \qquad \sigma \sim \mathrm{Exp}(1).$$

These priors do **not** come from the likelihood. They are modeling choices.

Here `normal(0, 10)` uses **standard deviation 10**, so it is a broad prior for μ on the scale of these data. `exponential(1)` uses **rate 1**, so

$$E(\sigma) = 1, \qquad P(\sigma > 0) = 1.$$

The exponential prior therefore supplies a simple positive prior for the unknown standard deviation.

Minimal example: translate the model into Stan

The paper model $\mu \sim N(0, 10^2)$, $\sigma \sim \text{Exp}(1)$, $Y_i \mid \mu, \sigma \sim N(\mu, \sigma^2)$ translates almost line by line:

```
data {
  int<lower=1> N;
  vector[N] y;
}
parameters {
  real mu;
  real<lower=0> sigma;
}
model {
  mu ~ normal(0, 10);
  sigma ~ exponential(1);
  y ~ normal(mu, sigma);
}
```

The first two lines in `model` are priors; the last line is the likelihood contribution.

Minimal example: translate the model into Stan

In this R Markdown file, `output.var="normal_stan"` is a knitr convenience: the Stan block is compiled and the resulting `stanmodel` object is stored as `normal_stan`.

In an ordinary R script, save the Stan code as, for example, `normal_model.stan`, and compile it explicitly with `stan_model()`.

RStan workflow: compile, then sample

```r
library(rstan)

# Save the preceding Stan program as normal_model.stan
normal_stan <- stan_model(file = "normal_model.stan")

stan_data <- list(
  N = length(y),
  y = as.vector(y)
)
fit <- sampling(
  object = normal_stan,
  data = stan_data,
  chains = 4,
  iter = 2000,
  warmup = 1000,
  seed = 20260812,
  cores = 4
)
```

RStan workflow

```
print(fit, pars = c("mu", "sigma"))
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##       mean se_mean   sd 2.5%  25%  50%  75% 97.5% n_eff Rhat
## mu    1.72       0 0.19 1.35 1.59 1.72 1.84  2.10  3118    1
## sigma 1.33       0 0.13 1.10 1.24 1.32 1.41  1.63  2940    1
##
## Samples were drawn using NUTS(diag_e) at Tue Oct  6 11:52:49 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

Two equivalent RStan workflows

1. Compile first, then sample:

```
sm <- stan_model(file = "normal_model.stan")
fit1 <- sampling(sm, data = stan_data, seed = 20260812)
```

2. Use the convenience wrapper `stan()`:

```
fit2 <- stan(
  file = "normal_model.stan",
  data = stan_data,
  seed = 20260812
)
```

Both compile the model and generate posterior draws. The first workflow is useful when we want to compile once and call `sampling()` repeatedly; `stan()` combines compilation and sampling in one call.

By default RStan uses NUTS; `algorithm = "HMC"` requests static HMC. Pass Stan's random seed through the `seed` argument rather than relying on R's `set.seed()`.

Bayesian linear regression in Stan

We use the built-in R data set `cars`, where `speed` is the car speed and `dist` is the stopping distance. We center `speed` around its sample mean before fitting the model:

$$x_i = \text{speed}_i - \overline{\text{speed}}, \qquad Y_i = \text{dist}_i.$$

Then

$$Y_i \mid \alpha, \beta, \sigma \sim N(\alpha + \beta x_i, \sigma^2).$$

Because `speed` is centered, α is the expected stopping distance at the average observed speed, while β is the expected change in stopping distance for a one-unit increase in speed. Centering changes the interpretation of the intercept, but not the slope. We use the illustrative priors

$$\alpha \sim N(0, 50^2), \qquad \beta \sim N(0, 10^2), \qquad \sigma \sim \text{Exp}(0.1).$$

Again these are **prior choices**, not quantities derived from the likelihood. The exponential rate 0.1 gives prior mean 10 for the residual standard deviation.

Bayesian linear regression in Stan

```
data {
  int<lower=1> N;
  vector[N] x;
  vector[N] y;
}
parameters {
  real alpha;
  real beta;
  real<lower=0> sigma;
}
model {
  alpha ~ normal(0, 50);
  beta ~ normal(0, 10);
  sigma ~ exponential(0.1);
  y ~ normal(alpha + beta * x, sigma);
}
```

Bayesian linear regression in Stan

The likelihood line corresponds directly to the product

$$\prod_{i=1}^{n} N(y_i \mid \alpha + \beta x_i, \sigma^2).$$

Here `output.var="reg_stan"` again makes knitr compile the block and store the resulting `stanmodel` object as `reg_stan`. In a regular R script we would save the block as a `.stan` file and call `stan_model()` explicitly.

Fit the regression with RStan

```
library(rstan)
# Save the preceding Stan program as regression_model.stan
reg_stan <- stan_model(file = "regression_model.stan")
x <- cars$speed - mean(cars$speed)
y <- cars$dist
stan_data_reg <- list(
  N = length(y),
  x = as.vector(x),
  y = as.vector(y)
)
fit_reg_stan <- sampling(
  object = reg_stan,
  data = stan_data_reg,
  chains = 4,
  iter = 2000,
  warmup = 1000,
  seed = 20260812,
  cores = 4
)
```

Fit the regression with RStan

```
print(
  fit_reg_stan,
  pars = c("alpha", "beta", "sigma"),
  probs = c(0.025, 0.5, 0.975)
)
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##        mean se_mean   sd  2.5%   50% 97.5% n_eff Rhat
## alpha 42.86    0.04 2.22 38.52 42.86 47.21  3691    1
## beta   3.93    0.01 0.42  3.12  3.92  4.76  3775    1
## sigma 15.56    0.03 1.59 12.73 15.46 19.14  3120    1
##
## Samples were drawn using NUTS(diag_e) at Tue Oct  6 11:54:07 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

Extract and inspect posterior draws

```
post <- extract(
  fit_reg_stan,
  pars = c("alpha", "beta", "sigma")
)
mean(post$beta)
```

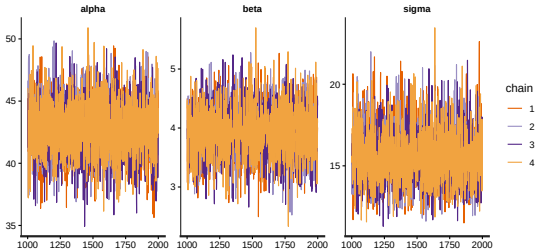
```
## [1] 3.925161
```

```
quantile(post$beta, c(0.025, 0.5, 0.975))
```

```
##     2.5%      50%    97.5%
## 3.115609 3.922527 4.757064
```

Extract and inspect posterior draws

```
traceplot(
  fit_reg_stan,
  pars = c("alpha", "beta", "sigma")
)
```



For serious analysis we also inspect effective sample sizes, $\widehat{R}$, divergences, and other HMC diagnostics.

RStan provides `check_hmc_diagnostics()` for diagnostics including divergences, tree depth and energy behaviour.

Important Stan diagnostics

Multiple chains should explore the same posterior distribution.

Important warnings/diagnostics include:

- $\widehat{R}$ substantially above 1;
- low effective sample size;
- divergent transitions;
- hitting maximum tree depth;
- problematic energy behaviour.

A divergence indicates that the numerical Hamiltonian trajectory is not adequately resolving the posterior geometry; it should not simply be ignored.

Warmup is not just burn-in

During Stan warmup, NUTS adapts algorithmic quantities such as the step size and metric.

Therefore warmup is not merely a set of unwanted early posterior draws.

After warmup, adaptation stops and the retained sampling phase is used for posterior inference.

RStan defaults to half of `iter` for warmup if `warmup` is not specified.

Thinning in Stan?

RStan supports a `thin` argument, but routine thinning is usually unnecessary for Stan's HMC/NUTS output. The RStan documentation explicitly notes that thinning is rarely necessary because HMC is efficient in effective samples per second.

Keep the draws unless storage or another specific constraint gives a reason not to.

Stan is not a substitute for modeling

Stan automates difficult computation, not statistical thinking.

We still need to decide:

- the likelihood;
- the prior;
- parameterization;
- whether the model is appropriate;
- what posterior quantity answers the scientific question.

A syntactically correct Stan program can still represent a poor statistical model.

Stan program anatomy

A useful first reading strategy is to separate the program into roles:

- `data`: quantities supplied by R and fixed during sampling;
- `parameters`: unknown quantities to infer;
- `transformed parameters`: deterministic functions stored with draws;
- `model`: prior and likelihood contributions;
- `generated quantities`: post-processing using each posterior draw.

The most important pedagogical connection is:

> The Stan model block is still just the statistical model we wrote on paper.

Stan's ~ notation

The statement

```
y ~ normal(mu, sigma);
```

adds the Normal log density to Stan's target log density.

Conceptually,

$$\texttt{target} \mathrel{+}= \sum_i \log p(y_i \mid \mu, \sigma).$$

Thus Stan does not avoid the likelihood. It provides a concise way to express it.

For learning, students should still be able to derive the likelihood on paper.

Generated quantities and posterior prediction

Posterior simulation is often used to generate replicated or future data.

For the regression model:

```
generated quantities {
  vector[N] y_rep;

  for (n in 1:N) {
    y_rep[n] = normal_rng(
      alpha + beta * x[n],
      sigma
    );
  }
}
```

Posterior predictive draws let us compare data generated by the fitted model with the observations.

This checks aspects of model adequacy that parameter summaries alone cannot reveal.

A practical Stan workflow

A useful workflow is:

1. write and inspect the statistical model;
2. derive/check the likelihood;
3. choose sensible parameter scales and priors;
4. compile and run multiple chains;
5. inspect $\widehat{R}$, ESS and HMC warnings;
6. inspect trace plots and posterior summaries;
7. perform posterior predictive checks;
8. revise the model when diagnostics or predictions reveal problems.

Computation is part of model criticism, not the final step after modeling.

Divergences and posterior geometry

Divergences often indicate regions where the numerical Hamiltonian dynamics cannot accurately follow the posterior geometry.

Common causes include:

- strong curvature;
- funnel-like geometries;
- poorly scaled parameters;
- difficult hierarchical parameterizations.

Simply increasing the number of iterations does not solve a geometric problem.

Reparameterization can be more important than running the sampler longer.

Centered and non-centered parameterizations

Hierarchical models often admit mathematically equivalent parameterizations with very different HMC geometry.

A generic centered form is

$$\theta_j \sim N(\mu, \tau^2).$$

A non-centered form writes

$$z_j \sim N(0, 1), \qquad \theta_j = \mu + \tau z_j.$$

The posterior distribution of scientifically meaningful quantities is unchanged, but the geometry seen by the sampler can change dramatically.

This is the HMC analogue of the regression-centering lesson from the previous lecture.

Comparing the algorithms

Method	Main idea	Strength	Typical weakness
Random-walk MH	propose + accept	general, simple	random-walk mixing
Gibbs	full conditionals	acceptance 1	correlated parameters
HMC	gradient dynamics	long efficient moves	tuning
NUTS	adaptive HMC path	less manual tuning	geometry still matters

Final perspective

MCMC is not one algorithm.

The common idea is to construct dependent draws whose stationary distribution is the target.

The progression in these lectures was:

$$\text{Bayes} \to \text{Markov chains} \to \text{Metropolis–Hastings} \to \text{Gibbs} \to \text{HMC/NUTS}.$$

For every method ask:

> Why is the target invariant, and how efficiently is it explored?

Stan makes sophisticated MCMC accessible, but the likelihood and model remain our responsibility.